

Aula EAD - Entrega da Lista 1

Algoritmos e Programação II

Marcos Henrique de Souza Almeida

“Resolver e entregar os Exercícios: 4.3; 5.4; 6.4; 7.8; 7.9; 11.5.4; 11.5.7; 11.5.10; 13.6.8; e 15.7.11.”

(Exercício 4.3)

```
#include <stdio.h>

struct Data {
    int dia;
    int mes;
    int ano;
};

int calcular_n(struct Data d) {
    int f_ano_mes;
    int g_mes;

    if (d.mes <= 2) {
        f_ano_mes = d.ano - 1;
        g_mes = d.mes + 13;
    } else {
        f_ano_mes = d.ano;
        g_mes = d.mes + 1;
    }

    int N = ((1461 * f_ano_mes) / 4) + ((153 * g_mes) / 5) + d.dia;

    return N;
}

int main() {
    struct Data d1, d2;

    printf("Digite a primeira data (dd mm aaaa): ");
    scanf("%d %d %d", &d1.dia, &d1.mes, &d1.ano);

    printf("Digite a segunda data (dd mm aaaa): ");
    scanf("%d %d %d", &d2.dia, &d2.mes, &d2.ano);

    int N1 = calcular_n(d1);
    int N2 = calcular_n(d2);

    int diferenca = N2 - N1;

    if (diferenca < 0) {
        diferenca = -diferenca;
    }

    printf("\nN1 = %d\n", N1);
    printf("N2 = %d\n", N2);
    printf("Dias de diferença entre as datas: %d\n", diferenca);

    return 0;
}
```

(Exercício 5.4)

```
#include <stdio.h>

#define TAM 10

struct Material {
    int codigo;
    char descricao[21];
    float quantidade;
};

int main() {
    struct Material galpao[TAM][TAM];

    int codigos_unicos[100];
    float soma_quantidade[100] = {0};
    int total_unicos = 0;

    printf("CADASTRO DOS MATERIAIS\n");

    for (int i = 0; i < TAM; i++) {
        for (int j = 0; j < TAM; j++) {
            printf("\nPosição [%d][%d]:", i, j);

            printf("Codigo: ");
            scanf("%d", &galpao[i][j].codigo);

            printf("Descrição: ");
            scanf(" %s", galpao[i][j].descricao);

            printf("Quantidade: ");
            scanf("%f", &galpao[i][j].quantidade);
        }
    }

    for (int i = 0; i < TAM; i++) {
        for (int j = 0; j < TAM; j++) {
            int cod = galpao[i][j].codigo;
            int encontrado = -1;

            for (int k = 0; k < total_unicos; k++) {
                if (codigos_unicos[k] == cod) {
                    encontrado = k;
                    break;
                }
            }

            if (encontrado != -1) {
                soma_quantidade[encontrado] += galpao[i][j].quantidade;
            } else {
                codigos_unicos[total_unicos] = cod;
                soma_quantidade[total_unicos] = galpao[i][j].quantidade;
                total_unicos++;
            }
        }
    }

    printf("\nQUANTIDADE TOTAL POR MATERIAL");

    for (int k = 0; k < total_unicos; k++) {
        printf("Codigo: %d | Quantidade Total: %.2f\n",
            codigos_unicos[k], soma_quantidade[k]);
    }

    return 0;
}
```

(Exercício 6.4)

```
#include <stdio.h>
#include <string.h>

#define MAX 100

struct torneio {
    char nome[50];
    int pontos;
};

void ordena(struct torneio equipe[MAX], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {

            int trocar = 0;

            if (equipe[j].pontos < equipe[j + 1].pontos) {
                trocar = 1;
            }
            else if (equipe[j].pontos == equipe[j + 1].pontos) {
                if (strcmp(equipe[j].nome, equipe[j + 1].nome) > 0) {
                    trocar = 1;
                }
            }
            if (trocar) {
                struct torneio aux = equipe[j];
                equipe[j] = equipe[j + 1];
                equipe[j + 1] = aux;
            }
        }
    }
}

int main() {
    struct torneio equipe[MAX];
    int n;

    printf("Digite o número de equipes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nEQUIPE %d:\n", i + 1);

        printf("Nome: ");
        scanf("%s", equipe[i].nome);

        printf("Pontos: ");
        scanf("%d", &equipe[i].pontos);
    }

    ordena(equipe, n);

    printf("\nRESULTADO DO TORNEIO\n");

    for (int i = 0; i < n; i++) {
        printf("%dº Lugar: %s | %d pontos\n", i + 1, equipe[i].nome, equipe[i].pontos);
    }
    return 0;
}
```

(Exercício 7.8)

```
#include <stdio.h>

//Letra (a)
int ciclo(int n) {
    int contador = 1;

    while (n != 1) {
        printf("%d ", n);

        if (n % 2 == 0) {
            n = n / 2;
        } else {
            n = 3 * n + 1;
        }
        contador++;
    }

    printf("\n");
    return contador;
}

// Letra (b)
int cicloR(int n) {
    printf("%d ", n);

    if (n == 1) {
        printf("\n");
        return 1;
    }

    if (n % 2 == 0) {
        return 1 + cicloR(n / 2);
    } else {
        return 1 + cicloR(3 * n + 1);
    }
}

// Letra (c)
int main() {
    int n;

    printf("Digite um numero inteiro positivo: ");
    scanf("%d", &n);

    if (n < 1) {
        printf("Por favor, digite um numero valido (n >= 1).\n");
        return 1;
    }

    printf("\nFunção Não-Recursiva\n");
    printf("Sequencia: ");

    int comp1 = ciclo(n);
    printf("Comprimento do ciclo: %d\n", comp1);

    printf("\nFunção Recursiva\n");
    printf("Sequencia: ");

    int comp2 = cicloR(n);
    printf("Comprimento do ciclo: %d\n", comp2);

    return 0;
}
```

(Exercício 7.9)

```
#include <stdio.h>

int potencia(int x, int n) {
    if (n == 0) {
        return 1;
    }

    if (n % 2 == 0) {
        int metade = potencia(x, n / 2);
        return metade * metade;
    }

    else {
        return x * potencia(x, n - 1);
    }
}

int main() {
    int a, b;

    printf("Digite a base (a): ");
    scanf("%d", &a);

    printf("Digite o expoente (b >= 0): ");
    scanf("%d", &b);

    if (b < 0) {
        printf("O expoente deve ser um inteiro não- negativo.\n");
        return 1;
    }

    int resultado = potencia(a, b);

    printf("\n%d^%d = %d\n", a, b, resultado);

    return 0;
}
```

(Exercício 11.5.4)

```
void trocas_sucessivas_rec(int v[], int n) {
    if (n <= 1) {
        return;
    }

    for (int i = 0; i < n - 1; i++) {

        if (v[i] > v[i + 1]) {
            int aux = v[i];
            v[i] = v[i + 1];
            v[i + 1] = aux;
        }
    }
    trocas_sucessivas_rec(v, n - 1);
}
```

(Exercício 11.5.7)

```
void selecao_rec(int v[], int n, int i) {
    if (i >= n - 1) {
        return;
    }

    int min = i;

    for (int j = i + 1; j < n; j++) {

        if (v[j] < v[min]) {
            min = j;
        }
    }

    if (min != i) {
        int aux = v[i];
        v[i] = v[min];
        v[min] = aux;
    }

    selecao_rec(v, n, i + 1);
}
```

(Exercício 11.5.10)

```
#include <stdio.h>

void insercao_rec(int v[], int n) {
    if (n <= 1) {
        return;
    }

    insercao_rec(v, n - 1);

    int chave = v[n - 1];
    int j = n - 2

    while (j >= 0 && v[j] > chave) {
        v[j + 1] = v[j];
        j--;
    }

    v[j + 1] = chave;
}
```

(Exercício 13.6.8)

```
#include <stdio.h>

#define MAX 1000

void intercala(int v[], int p, int q, int r) {
    int w[MAX];
    int i = p, j = q, k = 0;

    while (i < q && j < r) {
        if (v[i] <= v[j]) {
            w[k++] = v[i++];
        }
        else {
            w[k++] = v[j++];
        }
    }

    while (i < q) w[k++] = v[i++];

    while (j < r) w[k++] = v[j++];

    for (i = p; i < r; i++) {
        v[i] = w[i - p];
    }
}

void mergesort(int v[], int p, int r) {
    if (p < r - 1) {
        int q = (p + r) / 2;
        mergesort(v, p, q);
        mergesort(v, q, r);
        intercala(v, p, q, r);
    }
}
```

(Exercício 15.7.11)

```
#include <stdio.h>
#include <string.h>

int separa(char v[][50], int p, int r) {
    char pivo[50];
    strcpy(pivo, v[r]);

    int i = p - 1;
    char aux[50];

    for (int j = p; j < r; j++) {

        if (strcmp(v[j], pivo) <= 0) {
            i++;
            strcpy(aux, v[i]);
            strcpy(v[i], v[j]);
            strcpy(v[j], aux);
        }
    }

    strcpy(aux, v[i + 1]);
    strcpy(v[i + 1], v[r]);
    strcpy(v[r], aux);

    return i + 1;
}

void quicksort(char v[][50], int p, int r) {
    if (p < r) {
        int j = separa(v, p, r);
        quicksort(v, p, j - 1);
        quicksort(v, j + 1, r);
    }
}

int main() {
    char palavras[5][50] = {
        "Banana",
        "Abacaxi",
        "Uva",
        "Laranja",
        "Manga"
    };

    int n = 5;
    quicksort(palavras, 0, n - 1);

    printf("STRINGS ORDENADAS\n");

    for (int i = 0; i < n; i++) {
        printf("%s\n", palavras[i]);
    }

    return 0;
}
```